YOLOv11 Person Detector for Autonomous Vehicle Safety

Overview

This ROS 2 node implements a real-time pedestrian detection and tracking system using YOLOv11, designed for autonomous vehicle safety applications. The system detects pedestrians, tracks their movements in 3D space, predicts their future trajectories, and makes collision avoidance decisions for vehicle control.

Features

- **Real-time Person Detection**: Uses YOLOv11 for high-accuracy pedestrian detection
- **3D Position Tracking**: Projects 2D detections into 3D space using depth information
- **Multi-layer Filtering**:
  - Real-time position filtering with velocity-based prediction
  - Path-level spike removal using median absolute deviation (MAD)
  - Exponential smoothing for stable trajectories
- **Trajectory Prediction**: Predicts pedestrian paths up to 5 seconds into the future
- **Collision Detection**: Analyzes predicted paths for potential vehicle collisions
- **Safety Decision Making**: Outputs vehicle control commands (CONTINUE, CONTINUE_SLOW, STOP)
- **Visual Debugging**: Publishes annotated images and RViz markers for visualization

---

File Structure

`detect_node.py`

Description: Main ROS 2 node implementing the YOLOv11-based pedestrian detection, tracking, and collision prediction system.

Key Components:

- **YoloV11PersonDetector Class**: Primary node class inheriting from `rclpy.node.Node`
- **YOLO Model Integration**: Loads and runs YOLOv11 inference on RGB camera frames
- **Depth Processing**: Converts stereo depth images to 3D coordinates
- **Real-time Filtering**: `_filter_detection_position()` - Kalman-filter-like smoothing with velocity prediction
- **Path Smoothing**: `_smooth_path_and_predict()` - Multi-pass spike removal and trajectory prediction

- **Tracking System**: `_update_tracks()` - Associates detections across frames using nearest-neighbor matching
- **Collision Analysis**: `_time_to_collision()` - Calculates time-to-collision for pedestrian paths
- **Safety Logic**: `_safety_filter()` - Makes final vehicle control decisions

---

ROS 2 Topics

Subscribed Topics

Topic Name	Message Type	Description	QoS Settings
`/oak/rgb/image_raw`	`sensor_msgs/Image`	RGB camera feed from OAK-D camera. Used for YOLO person detection. Encoding: BGR8	Reliability: BEST_EFFORT, History: KEEP_LAST, Depth: 5
`/oak/stereo/image_raw`	`sensor_msgs/Image`	Stereo depth image from OAK-D camera. Provides depth information (in mm) for 3D projection. Converted to meters internally.	Reliability: BEST_EFFORT, History: KEEP_LAST, Depth: 5

Published Topics

Topic Name	Message Type	Description	QoS Settings
`detections`	`vision_msgs/Detection2DArray`	2D bounding box detections of all people found in the current frame. Includes confidence scores, class IDs, and bounding box dimensions.	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10
`detections/image`	`sensor_msgs/Image`	Debug visualization image showing bounding boxes, crossing indicators, and tracking paths overlaid on the camera feed. Color-coded by track ID.	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10
`vehicle_control_command`	`std_msgs/String`	Vehicle control command string. Possible values: **"CONTINUE"** (safe to proceed), **"CONTINUE_SLOW"** (proceed with caution), **"STOP"** (imminent collision risk).	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10

Topic Name	Message Type	Description	QoS Settings
`person_marker`	`visualization_msgs/Marker`	RViz sphere marker showing the current 3D position of each tracked person. Green spheres (0.3m diameter) in the `base_link` frame.	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10
`person_path`	`visualization_msgs/Marker`	RViz line strip marker showing the smoothed historical path of each tracked person. Yellow lines representing the trajectory traveled.	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10
`person_prediction`	`visualization_msgs/Marker`	RViz line strip marker showing the predicted future path of each person (5 seconds ahead). Red lines indicating predicted trajectory.	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10
`car_marker`	`visualization_msgs/Marker`	RViz cube marker representing the ego vehicle at the origin of the `base_link` frame. Blue cube (0.5m x 0.5m x 0.5m).	Reliability: RELIABLE, History: KEEP_LAST, Depth: 10

Transform (TF) Broadcasting

The node publishes a static transform from `base_link` to `oak_rgb_camera_optical_frame`:

- **Translation**: (0.535, 0.0, 1.683) meters
- **Rotation**: 90° pitch rotation (camera looking forward)
- **Purpose**: Converts camera optical frame coordinates to vehicle base frame coordinates

---

Parameters

Model Configuration

Parameter	Type	Default	Description
image_topic	string	/oak/rgb/image_raw	Input RGB camera topic
depth_topic	string	/oak/stereo/image_raw	Input depth camera topic
model_path	string	yolo11n.pt	Path to YOLOv11 model weights file
device	string	cuda:0	Inference device (cuda:0, cuda:1, cpu)
conf	float	0.35	Confidence threshold for detections (0.0-1.0)
iou	float	0.45	IoU threshold for non-maximum suppression
imgsz	int	640	Input image size for YOLO model
half	bool	false	Use FP16 half-precision inference (faster, less accurate)
max_detections	int	100	Maximum number of detections per frame

Visualization & Debug

Parameter	Type	Default	Description
publish_debug_image	bool	true	Enable/disable publishing annotated debug images

Safety & Tracking

Parameter	Type	Default	Description
vehicle_speed	float	0.5	Current vehicle speed in m/s (for collision calculations)
min_safe_distance	float	1.0	Minimum safe distance from pedestrians (meters)
max_ped_speed_for_safety	float	2.0	Max pedestrian speed threshold for safety decisions (m/s)
x_vel_buffer_length	int	3	Buffer size for X-velocity smoothing
y_vel_buffer_length	int	10	Buffer size for Y-velocity smoothing

Internal Constants (Hardcoded)

- **position_smooth_alpha**: 0.3 - Real-time position filtering smoothing factor
- **smooth_alpha**: 0.6 - Track position exponential smoothing factor
- **max_path_len**: 100 - Maximum number of historical points per track
- **max_missing_frames**: 10 - Frames before track is deleted
- **prediction_time**: 5.0 seconds - Future prediction time horizon

- `prediction_points`: 20 - Number of points in predicted trajectory

Algorithm Details

1. Detection Pipeline

```
RGB Image → YOLOv11 → Bounding Boxes → Depth Lookup → 3D Coordinates
```

- YOLOv11 detects people (COCO class 0) in RGB images
- Depth value sampled at bounding box center
- 3D position computed using pinhole camera model

2. Real-time Position Filtering (Layer 1)

Method: Velocity-based prediction with spike rejection

```python
predicted_position = previous_position + velocity × dt
deviation = |new_position - predicted_position|

if deviation > max_reasonable_distance:
    use predicted_position  # Reject spike
else:
    filtered_position = α × new_position + (1-α) × predicted_position
```

- **Spike Threshold**: 5.0 m/s maximum pedestrian speed
- **Smoothing Alpha**: 0.3 (30% new data, 70% prediction)
- **Velocity Smoothing**: Exponential smoothing on velocity estimates

3. Path-level Filtering (Layer 2)

Multi-pass spike removal:

1. **Pass 1 - Statistical Outlier Removal**:
   - Uses Median Absolute Deviation (MAD)
   - Replaces points >3 MAD from median
2. **Pass 2 - Median Filtering**:
   - 5-point median filter window

- Removes isolated spikes

3. **Pass 3 - Geometric Spike Detection**:
   - Checks deviation from linear interpolation (threshold: 0.3m)
   - Checks angle between consecutive segments (threshold: 60°)
   - Replaces spikes with interpolated values

Smoothing:

- 15-point moving average
- Exponential smoothing (α=0.15)

4. Trajectory Prediction

Velocity Estimation:

- Uses last 60 points (3× prediction points)
- Outlier filtering using 2-sigma rule
- Clamps maximum speed to 3.0 m/s
- Projects velocity onto ground plane (X=0)

Prediction:

- Constant velocity model: $\boxed{\text{position}(t) = \text{position}_0 + \text{velocity} \times t}$
- 20 prediction points over 5 seconds
- Maximum prediction distance: 5.0 meters

5. Collision Detection

Vehicle Geometry:

- Width: 1.0 meter
- Length: 2.0 meters
- Positioned at origin in base_link frame

Collision Criteria:

- Path intersects vehicle bounding box
- Distance to vehicle path < 0.3 meters
- Time-to-collision (TTC) calculated for each track

6. Safety Decision Logic

```python
if TTC < 1.0 second:
    → STOP (imminent danger)
elif pedestrian_speed > 2.0 m/s AND not slowing:
    → STOP (fast-moving, unpredictable)
elif pedestrian_slowing AND TTC > 3.0 seconds:
    → CONTINUE_SLOW (pedestrian yielding)
elif not_slowing AND TTC < 3.0 seconds:
    → STOP (pedestrian not yielding)
else:
    → CONTINUE_SLOW (default caution)
```

Installation

Dependencies

```bash
# ROS 2 (Humble recommended)
sudo apt install ros-humble-desktop

# Python packages
pip install ultralytics opencv-python numpy

# ROS 2 Python packages
pip install cv-bridge rclpy sensor-msgs vision-msgs visualization-msgs
```

YOLOv11 Model

Download YOLOv11 weights:

```bash
# Nano model (fastest)
wget https://github.com/ultralytics/assets/releases/download/v0.0.0/yolo11n.pt

# Or use other variants: yolo11s.pt, yolo11m.pt, yolo11l.pt, yolo11x.pt
```

Usage

Launch the Node

```bash
# Source ROS 2 workspace
source /opt/ros/humble/setup.bash
source ~/your_workspace/install/setup.bash

# Run with default parameters
ros2 run yolo_person_detector detect_node

# Run with custom parameters
ros2 run yolo_person_detector detect_node \
  --ros-args \
  -p image_topic:=/camera/image_raw \
  -p depth_topic:=/camera/depth/image_raw \
  -p model_path:=/path/to/yolo11m.pt \
  -p device:=cuda:0 \
  -p conf:=0.5
```

Visualize in RViz

```bash
# Launch RViz
rviz2

# Add displays:
# - Image: /detections/image
# - Marker: /person_marker
# - Marker: /person_path
# - Marker: /person_prediction
# - Marker: /car_marker
# - TF
```

Monitor Vehicle Commands

```bash
ros2 topic echo /vehicle_control_command
```

Performance

- **Inference Speed**: ~30 FPS on NVIDIA RTX 3080 (with yolo11n.pt)
- **Tracking Latency**: <5ms per frame
- **Detection Range**: Up to 10 meters (depth-dependent)
- **Maximum Tracked Pedestrians**: 100 simultaneous tracks

Coordinate Frames

Camera Optical Frame (`oak_rgb_camera_optical_frame`)

- **X**: Right
- **Y**: Down
- **Z**: Forward (depth)

Vehicle Base Frame (`base_link`)

- **X**: Up (vertical)
- **Y**: Forward (driving direction)
- **Z**: Left (lateral)

Debugging

Enable Verbose Logging

The node logs FPS every 30 frames and safety decisions in real-time:

```bash
# View logs
ros2 run yolo_person_detector detect_node 2>&1 | grep -E "(FPS|Safety Filter|Collision)"
```

Common Issues

1. **No detections**: Check camera topics with `ros2 topic echo /oak/rgb/image_raw`
2. **Low FPS**: Reduce `imgsz` parameter or use smaller model (yolo11n.pt)
3. **Noisy paths**: Decrease `position_smooth_alpha` for more aggressive filtering

4. **False collision warnings**: Adjust `min_safe_distance` and TTC thresholds

---

Color Coding (Debug Image)

- **Green Box**: Person detected, safe
- **Orange Box**: Person in caution zone
- **Red Box**: Person crossing vehicle path
- **Magenta/Cyan/Orange/Yellow Lines**: Track paths (color by track ID)

Future Enhancements

- [ ] Multi-camera fusion
- [ ] Deep learning-based trajectory prediction (LSTM/Transformer)
- [ ] Intention recognition (stopping vs. crossing)
- [ ] Integration with vehicle planning stack
- [ ] GPU-accelerated filtering pipeline

Citation

If you use this code in your research, please cite:

```bibtex
@software{yolov11_person_detector,
  title = {YOLOv11 Person Detector for Autonomous Vehicles},
  author = {Your Name},
  year = {2024},
  url = {https://github.com/your-repo}
}
```

License

This project is licensed under the MIT License.

Contact

For questions or issues, please open an issue on GitHub or contact: your.email@example.com